Simulador Strict 2PL

Controle de acesso aos dados de um banco

Pedro Lopes de Macedo
Allan Pedroso Cunico

Desenvolvido em C# com .NET 10

CARACTERÍSTICAS

O que o simulador faz e o que não faz?

Implementadas

- Locks S e X, com upgrade S → X
- Strict 2PL: locks até o commit ou abort
- Delay com fila por transação
- Deadlock: detecção, rollback e reinício
- HF, locks e delay passo a passo
- Interface gráfica e console

Não implementadas

- Execução concorrente real (é sequencial)
- Prevenção de deadlock (só detecção)
- Abort manual na HI
- Fila justa de locks por dado
- Persistência em disco (só memória)

DEADLOCKS

Como tratamos um deadlock?

1. Detectar

Monta um grafo de espera: T1 → T2 se T1 espera um lock de T2.

Uma busca em profundidade (DFS) procura ciclos.

2. Escolher a vítima

A transação de maior número dentro do ciclo.

3. Abortar e refazer

Rollback com o log de valores anteriores, em ordem inversa.

Remove as operações da HF, libera os locks e recoloca a transação inteira no fim da HI.

EXEMPLO

```
HI: w1[x,10] w2[y,20] w1[y,11] w2[x,21] c1 c2
```

T1 espera Y (de T2) e T2 espera X (de T1): ciclo. T2 é a vítima, refaz no fim e a HF termina sem deadlock.

ENTRADA DE DADOS

Como digitar a história inicial?

UMA LINHA, OPERAÇÕES SEPARADAS POR ESPAÇO

```
r1[x] w1[x,5] r2[x] w2[y] c1 c2
```

Formato

- r = leitura, w = escrita, c = commit
- O número depois da letra é a transação
- Colchetes ou parênteses: r1[x] ou r1(x)
- w1[x,5] grava 5; w1[x] soma 1 ao valor

Regras

- Todos os dados começam em 0
- Cada transação tem um commit, no final
- A entrada é validada antes de executar
- Locks e unlocks não são digitados: o simulador insere

SAÍDA DE DADOS

O que o simulador mostra?

Passo a passo

- Cada operação enviada e o lock concedido
- Operações em delay e quem as bloqueia
- Commit, unlocks e reavaliação do delay
- Deadlock: ciclo, vítima, rollback e reinserção
- A HF crescendo, e perdendo as operações removidas

Resultado final

- História final (HF)
- Valor final de cada dado
- Número de deadlocks
- Se terminou ou ficou incompleta (pendências, delay, locks ativos)

EXEMPLO DO DEADLOCK

```
HF: w1[X,10] w1[Y,11] c1 w2[Y,20] w2[X,21] c2
X = 21 Y = 20 Deadlocks: 1
```

ESTRUTURAS DE DADOS

Como tudo é armazenado?

Estruturas

Transações: lista de *Operacao* por transação (para refazer inteira)

Histórias: HI e HF são listas; uma cópia guarda a entrada

Controle: fila de delay (Queue) por transação, log de undo, grafo de espera

Dados: SortedDictionary nome → valor, iniciando em 0

Tabela de locks

Um LockInfo por dado: lista de leitores (S) e um escritor (X)

Concessão: S se não há outro escritor; X se não há outro leitor nem escritor; upgrade S → X quando só a própria transação lê

Liberação: só no commit ou abort (no abort, rollback primeiro); depois o delay é reavaliado

EXEMPLO: TABELA APÓS w1[x,10] r2[y]

```
x → escritor T1
y → leitores T2
```